

编译原理实验一实验报告

姓名： 郑彦文 学号： 2022112879

1 已实现功能

本实验实现了一个基于 GNU Flex 与 GNU Bison 的 C- 语言前端程序，能够对输入源文件完成词法分析、语法分析，并在无词法/语法错误时输出语法树。程序入口为 `main.c`，词法规则位于 `lexical.l`，语法规则位于 `syntax.y`，语法树结构与打印逻辑位于 `tree.h/tree.c`。整体流程为：读取源文件 → 调用 `yyparse()` → 由 Bison 驱动 Flex 逐个读取 token → 无错时输出语法树。

1.1 词法分析功能

程序能够识别实验要求中的关键字、标识符、整数、浮点数、关系运算符、逻辑运算符、界符和括号等基本词法单元，并在识别时同步构造语法树叶子结点。对于词法错误，程序按照实验要求输出 `Error type A at Line n: ...`。词法分析中使用了两类状态：`COMMENT` 用于处理块注释，`SKIPLINE` 用于在某一出现词法错误后跳过该行剩余内容，从而避免同一行重复报错。

本程序额外完成了实验一的扩展要求：

- 识别八进制与十六进制整数，并对非法形式如 `09`、`0x3G` 报错；
- 识别指数形式浮点数，并兼容测试中出现的 `1.05e-4` 与 `1.05e - 4` 两种写法；
- 识别 `//` 单行注释和 `/* ... */` 块注释，并对未终止块注释给出错误提示。

1.2 语法分析与语法树构造

程序按照 C- 文法实现了外部定义、结构体声明、函数定义、变量定义、数组定义、表达式、条件语句、循环语句、返回语句和函数调用等语法结构。具体支持的典型形式包括：全局变量定义、无参/带参函数定义、局部变量定义、结构体类型引用、数组访问、点号访问、赋值表达式、逻辑表达式、函数调用表达式等。Bison 归约时，程序为每个非终结符创建树结点，并将右部符号依次挂接为其孩子，从而构造满足实验要求的多叉语法树。

语法树采用“长子-兄弟”表示法。该结构一方面便于按照归约顺序插入孩子结点，另一方面也为实验二继续扩展语义分析保留了空间。最终输出时，程序按先序遍历打印语法树，并严格遵守实验手册规定的缩进与附加信息格式。

1.3 错误处理

程序实现了词法错误 A 与语法错误 B 的区分输出。词法分析阶段通过记录最近报错行号避免同一行重复报错；语法分析阶段结合 Bison 的 `error` 记号实现了若干定向恢复规则，能够对典型错误给出较明确的提示，例如：

- 数组访问中的 `a[5,3]` 可报告 `Missing "]"`；
- `if (...) i = 1 else ...` 可报告 `Missing ";"`；
- 部分缺失右括号、右中括号、分号的场景可继续分析后续输入。

同时，程序满足实验要求：一旦存在词法错误或语法错误，则不输出任何语法树内容。当前实现已经可以正确通过给定测试集，并能够对实验手册中的核心样例给出符合要求的输出。

2 编译与运行方法

本项目提供了 `Makefile`，推荐在 Linux 环境下直接使用 `make` 编译。实验环境中只依赖 GCC、Flex、Bison 及标准 C 库，不使用额外第三方库。

2.1 编译步骤

在 `expr1/code` 目录下执行：

```
make clean
make
```

上述命令会自动完成以下工作：

- 使用 `bison -d -v syntax.y` 生成 `syntax.tab.c`、`syntax.tab.h` 和 `syntax.output`；
- 使用 `flex lexical.l` 生成 `lex.yy.c`；
- 使用 `gcc` 将生成文件和手写源文件链接为可执行程序 `cc`。

若不使用 `Makefile`，也可手动执行：

```
bison -d -v syntax.y
flex lexical.l
gcc -Wall -Wextra -std=gnu11 -o cc lex.yy.c syntax.tab.c tree.c main.c -lfl
```

2.2 运行方式

对于单个输入文件，可执行：

```
./cc test/03_correct_inc.cmm
```

若要批量运行全部测试，可执行：

```
./run_tests.sh
```

该脚本会先重新编译工程，再顺序运行 `test/*.cmm`，并将每个用例的输出保存到 `test/output/` 目录。

3 实现亮点与设计说明

首先是本程序实现了对二义性的处理。由于表达式文法天然存在二义性，比如 `a + b * c` 需要保证乘法优先于加法；`if-else` 语句还存在悬空 `else` 问题。对此，本程序在 `syntax.y` 中显式声明了运算符优先级与结合型，并使用 `%prec LOWER_THAN_ELSE` 消除悬空的 `else` 冲突。这一部分对应实验手册中对 Bison 冲突处理的要求，也是本实验中核心的语法分析设计点之一。

其次是实验一要求不仅要判断“是否有错”，还需要尽可能输出正确的错误类型与行号。对此，本程序没有只依赖 Bison 默认的 `syntax error`，而是在若干高频错误点加入了定向恢复规则，使程

序既能够输出更接近样例的错误说明，也能在一处报错之后继续分析后续代码，便于一次性发现多个错误。这里的实现思路是：把恢复点放在分号、右括号、右中括号等边界处，可以减少无关 token 的丢弃，又能保持后续继续分析。

最后是本程序的语法树结构可以扩展。实验一只要求输出语法树，但是本程序在实现时仍然将树结构与打印逻辑独立封装。语法树使用统一的 `TreeNode` 结构保存节点名、行号、token、附加信息以及兄弟/孩子关系；词法分析与语法分析共享该结构，这样就避免了中间转换。这样做的好处是，实验二需要语义分析的话，可以直接复用这棵树进行，而不需要重新设计前端数据结构，具有后续可扩展性。